

1. K-Nearest Neighbors (KNN)

Definition

KNN (K-Nearest Neighbors) is a **Supervised Machine Learning Algorithm** used for both classification and regression.

It classifies a new data point based on the majority class of its nearest neighbors.

How KNN Works

1. Choose value of K.
 2. Calculate distance between new point and all training points.
 3. Find K nearest neighbors.
 4. Count classes of neighbors.
 5. Assign majority class to new data point.
-

Daily Life Example

Suppose a new student joins a class.

Nearby students:

Student Favorite Subject

A	Math
B	Math
C	Science
D	Math
E	Science

If $K = 5$

Math = 3

Science = 2

New student is classified as **Math Lover**.

Distance Formula

KNN generally uses Euclidean Distance:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Advantages

- Easy to understand
 - No training phase
 - Works well for small datasets
-

Disadvantages

- Slow for large datasets
 - Sensitive to irrelevant features
 - Requires feature scaling
-

KNN Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Model
model = KNeighborsClassifier(n_neighbors=3)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

2. Naive Bayes

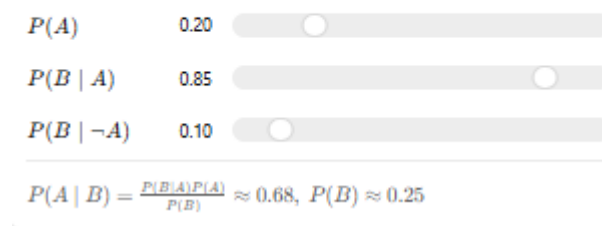
Definition

Naive Bayes is a supervised machine learning algorithm based on **Bayes Theorem**.

It assumes all features are independent of each other.

Bayes Theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$



Where:

- $P(A|B)$ = Posterior Probability
 - $P(B|A)$ = Likelihood
 - $P(A)$ = Prior Probability
 - $P(B)$ = Evidence
-

How Naive Bayes Works

Suppose an email contains words:

- Free
- Offer
- Win

The algorithm calculates probability of:

- Spam
- Not Spam

Whichever probability is higher becomes the prediction.

Daily Life Example

Imagine a doctor predicting flu.

Symptoms:

- Fever
- Cough
- Headache

Naive Bayes calculates:

Probability(Flu | Symptoms)

If probability is high $\rightarrow$ Flu.

Types of Naive Bayes

1. Gaussian Naive Bayes

Used for continuous values.

Example:

- Age
 - Salary
 - Height
-

2. Multinomial Naive Bayes

Used for text classification.

Example:

- Spam detection
 - Sentiment analysis
-

3. Bernoulli Naive Bayes

Used for binary features.

Example:

- Yes/No
- True/False

Advantages

- Very fast
- Works well on text data
- Requires less training data

Disadvantages

- Assumes feature independence
- Lower performance on complex datasets

Naive Bayes Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load Dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model = GaussianNB()

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

3. Random Forest

Definition

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees and produces a final prediction through voting.

It is used for:

- Classification
 - Regression
-

Why Random Forest?

A single decision tree may overfit.

Random Forest creates many trees and combines them to improve accuracy.

How Random Forest Works

1. Create multiple random datasets.
 2. Build multiple decision trees.
 3. Each tree makes a prediction.
 4. Majority voting gives final output.
-

Daily Life Example

Suppose 10 doctors diagnose a patient.

Predictions:

- 7 Doctors → Disease
- 3 Doctors → No Disease

Final Decision:

Disease

This is similar to Random Forest voting.

Random Forest Architecture

Dataset
|

```
V
Tree 1 ---> Prediction A
Tree 2 ---> Prediction A
Tree 3 ---> Prediction B
Tree 4 ---> Prediction A
Tree 5 ---> Prediction A
```

Final Output = A

Advantages

- High Accuracy
 - Reduces Overfitting
 - Handles Large Datasets
 - Works with Missing Values
-

Disadvantages

- Slow Training
 - More Memory Usage
 - Harder to Interpret
-

Random Forest Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train
model.fit(X_train, y_train)

# Prediction
```

```
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Comparison Table

Feature	KNN	Naive Bayes	Random Forest
Type	Supervised	Supervised	Supervised
Training Speed	Very Fast	Fast	Medium
Prediction Speed	Slow	Fast	Fast
Accuracy	Medium	Medium	High
Handles Large Data	No	Yes	Yes
Overfitting	Low	Low	Very Low
Best Use Case	Pattern Matching	Text Classification	General Purpose ML

Why is Random Forest generally better than a single Decision Tree?

Answer: Because Random Forest combines predictions from multiple trees, reducing overfitting and improving accuracy, stability, and generalization on unseen data.

Real-World Applications

- **KNN:** Recommendation systems, handwriting recognition.
- **Naive Bayes:** Spam email detection, sentiment analysis.
- **Random Forest:** Loan approval, fraud detection, disease prediction, customer churn prediction.